University of Jeddah
College of Computer Science and Engineering
Department of Software Engineering

CCSW 323 · SOFTWARE TESTING & VALIDATION

Class Project Report

Coverage-Based Testing of a Java Student Management System

Software Under Test

Student Management System (Java)

Prepared by

#	Name	ID
1	Enas Hamed Alqarni	2314104
2	Joury Ghorab	2312080
3	Jana Akkad	2313200

Instructor
Dr. Mona Altherwi

2. Software Under Test

2.1 Brief Description

The Student Management System (SMS) is a Java console application for managing a roster of students. It supports adding, searching, updating, deleting, sorting, filtering, CSV/TXT import-export, merging two systems, and summary reports (top-5 per year, failing students). Each record holds a name, ID, GPA, year, and department.

We chose this codebase because it is fully backend, uses only the Java standard library, and has enough decisions, loops, and switches to apply the three coverage models we needed.

2.2 GitHub Repository

https://github.com/Mohammed-3tef/Student_Management_System.git

2.3 Size of the Software

The project is roughly one thousand lines of Java across six classes. Most of the logic sits in StudentSystem and InputValidator, which is where we focused the test cases.

Class	Lines of Code	Number of Methods	Tested Here?
App.java	260	1 (main)	indirectly
Student.java	20	2 (constructors)	no
StudentSystem.java	327	13	yes — 3 functions
InputValidator.java	137	9	indirectly
CsvFileHandler.java	93	3	no
TxtFileHandler.java	130	1	no
TOTAL	≈ 967	29	—

Three functions inside StudentSystem drive every test case in Section 4:

- displayTop5() — used to apply Logic Coverage on its inner predicate.
- mergeStudentSystem(StudentSystem) — used to apply Input Space Partitioning.
- removeStudentByID(int) — used to apply Graph Coverage, including Prime Path Coverage.

3. Selected Model and Coverage Criteria

The project requires at least two testing models with two criteria each. We used three so each major family from the course is covered, and each model targets a different function from Section 2.

3.1 Three Testing Models

#	Model	Function under test	What the model views
1	Logic Coverage	displayTop5()	Boolean predicate and its three clauses
2	Input Space Partitioning	mergeStudentSystem(StudentSystem)	The structure of the input domain
3	Graph Coverage	removeStudentByID(int)	The control flow graph of the function

3.2 Two Coverage Criteria per Model

For every model we picked one weaker criterion and one stronger criterion that extends it, so the report shows both ends of the cost-rigour scale. The pairs are listed below.

Model	Criterion 1 (basic)	Criterion 2 (stronger / different)
Logic Coverage	Predicate Coverage (PC)	Correlated Active Clause Coverage (CACC)
Input Space Partitioning	Each Choice Coverage (ECC)	Base Choice Coverage (BCC)
Graph Coverage	Node Coverage (NC)	Prime Path Coverage (PPC)

3.3 How Many Test Cases?

Five test cases per criterion, thirty in total. No two share the same input or scenario. Section 4 lists each one with input, expected output, actual output, and verdict.

4. Test Cases

For each model we identify the function under test, apply both criteria, and list the test cases with their input, expected output, actual output, and verdict. Five cases per criterion, thirty total, no repeats.

4.1 Logic Coverage on displayTop5()

▪ Function under test

displayTop5() prints, for each academic year, up to five passing students (GPA ≥ 2.0) ranked by GPA. Each candidate is screened by a single conjunctive predicate, which is what we target.

▪ Source code

```
public void displayTop5() {
    String[] years = {"First", "Second", "Third", "Fourth"};
    for (int numberOfYears = 0; numberOfYears < years.length; numberOfYears++) {
        System.out.println("\n" + years[numberOfYears] + " Year:");
        sortStudentsBy("GPA");
        int count = 1;
        for (Student student : studentList) {
            if (student.year.equals(years[numberOfYears]) && count <= 5 && student.GPA
>= 2.0) {
                System.out.println(count + ". " + student.name + " | ID " + student.ID
+ " | GPA " + student.GPA);
                count++;
            }
        }
    }
}
```

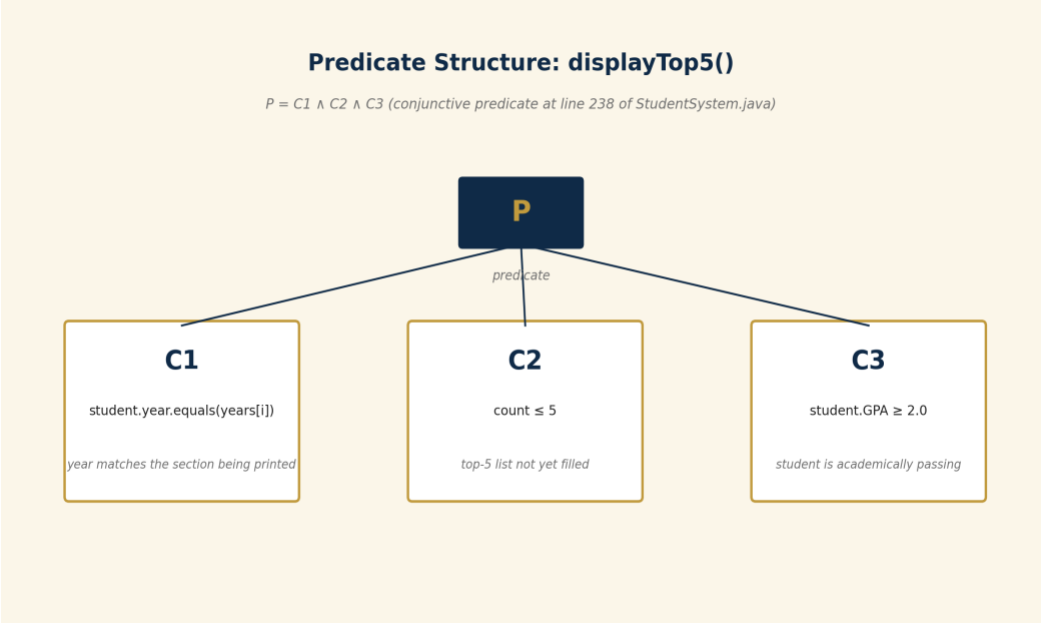
▪ Predicate

```
student.year.equals(years[numberOfYears])  &&  count <= 5  &&  student.GPA >= 2.0
```

▪ Clauses

- C1 : student.year matches the year currently being printed.
- C2 : count <= 5 — the slot for the year being printed is not yet full.
- C3 : student.GPA >= 2.0 — the student is academically passing.

Predicate definition: P = C1 ∧ C2 ∧ C3



Criterion 1 Predicate Coverage (PC)

PC requires the predicate to take both T and F at least once. We use five truth-table rows so the predicate flips for different reasons, not just one.

Truth Table for P = C1 ∧ C2 ∧ C3 (complete enumeration of $2^3 = 8$ rows)

All 8 truth-value combinations for the three clauses are shown below. P is true only when all three clauses are true (row 1). The five highlighted rows are the ones we picked to satisfy PC and CACC: row 1 is the all-true anchor, rows 2–4 each flip exactly one clause (active clauses for CACC), and row 5 combines two failing clauses. The remaining three rows are subsumed by these because they do not add any new active-clause behaviour.

#	C1	C2	C3	P	Used by test
1	T	T	T	T	TC-LC-PC-01 / CACC anchor
2	F	T	T	F	TC-LC-PC-02 (C1 active for CACC-02)
3	T	F	T	F	TC-LC-PC-03 (C2 active for CACC-03)
4	T	T	F	F	TC-LC-PC-04 (C3 active for CACC-04)
5	F	T	F	F	TC-LC-PC-05 (two clauses false)
6	F	F	T	F	— (subsumed)
7	T	F	F	F	— (subsumed)
8	F	F	F	F	— (subsumed)

PC Test Requirements (clause values per test case)

PC requires the predicate P to be both true and false at least once. The table below makes explicit which clause values each test case exercises and the resulting value of P. Across the five tests, P takes both true (row 1) and false (rows 2–5), so PC is satisfied.

Test Case ID	C1	C2	C3	P	Note
TC-LC-PC-01	T	T	T	**T**	All clauses true → P is true
TC-LC-PC-02	**F**	T	T	**F**	C1 flipped → P flips to false
TC-LC-PC-03	T	**F**	T	**F**	C2 flipped → P flips to false
TC-LC-PC-04	T	T	**F**	**F**	C3 flipped → P flips to false
TC-LC-PC-05	F	T	F	**F**	Two clauses false → P false

Test Cases (PC)

Test Case ID	Input	Expected Output	Actual Output	Result
TC-LC-PC-01	Aisha, ID 1001, GPA 3.85, year=First (lone First-year student)	Aisha appears in the First Year section (P=T)	Aisha appears in the First Year section	Pass
TC-LC-PC-02	Bilal, ID 1002, GPA 3.40, year=Second (lone Second-year student); we inspect the First Year section	Bilal does NOT appear in the First Year section (C1 false)	Bilal does not appear in the First Year section	Pass
TC-LC-PC-03	Six First-year students with passing GPAs (3.95, 3.85, 3.75, 3.65, 3.55, 3.10); we look at the sixth (Cyrus)	Cyrus is filtered out because the top-5 slots are full (C2 false)	Cyrus is not printed; the other five appear	Pass
TC-LC-PC-04	Dina, ID 1004, GPA 1.50, year=First (failing GPA)	Dina is hidden because she fails C3	Dina is hidden	Pass
TC-LC-PC-05	Faisal, ID 1005, GPA 1.30, year=Third (Third-year and failing simultaneously)	Faisal is absent from both First Year (C1 false) and Third Year (C3 false) sections	Faisal is absent from both sections	Pass

Criterion 2 Correlated Active Clause Coverage (CACC)

CACC is stronger than PC: each Ci must independently determine P. For an AND predicate, we hold the other two clauses true and toggle Ci. The fifth case combines two failing clauses to confirm each one alone flips the predicate.

CACC Test Requirements (active clause per test case)

CACC requires each clause Ci to independently determine P. For an AND predicate this means holding the other two clauses true and toggling Ci. The active clause for each test is highlighted in gold below. Together rows CACC-01 through CACC-04 prove every clause can independently flip P; CACC-05 confirms multiple failing clauses still produce P = F.

Test Case ID	C1	C2	C3	P	Active clause
TC-LC-CACC-01	T	T	T	T	Anchor: all clauses true
TC-LC-CACC-02	F	T	T	F	C1 active: flipping C1 flips P
TC-LC-CACC-03	T	F	T	F	C2 active: flipping C2 flips P
TC-LC-CACC-04	T	T	F	F	C3 active: flipping C3 flips P
TC-LC-CACC-05	F	T	F	F	Two clauses false simultaneously

Test Cases (CACC)

Test Case ID	Input	Expected Output	Actual Output	Result
TC-LC-CACC-01	Hala, ID 3001, GPA 3.95, year=Second (CACC anchor: all clauses true)	Hala is printed in the Second Year section	Hala is printed in the Second Year section	Pass
TC-LC-CACC-02	Idris, ID 3002, GPA 3.60, year=Fourth; we inspect the Second Year section	C1 active: Idris is hidden in Second Year but printed in Fourth Year	Idris hidden in Second Year, printed in Fourth Year	Pass
TC-LC-CACC-03	Five top Third-year students plus Jana (sixth, GPA 3.50)	C2 active: Jana is filtered out, the five top students remain	Jana hidden, top five present	Pass
TC-LC-CACC-04	Khaled, ID 4007, GPA 1.95, year=Fourth (sub-2.0)	C3 active: Khaled is hidden in the Fourth Year section	Khaled is hidden	Pass
TC-LC-CACC-05	Five top First-year students; then Layla (failing GPA) and Mira (sixth First-year, GPA 2.50)	Layla suppressed by C3, Mira suppressed by C2; the top five remain	Layla suppressed, Mira suppressed, top five present	Pass

4.2 Input Space Partitioning on mergeStudentSystem(StudentSystem)

▪ Function under test

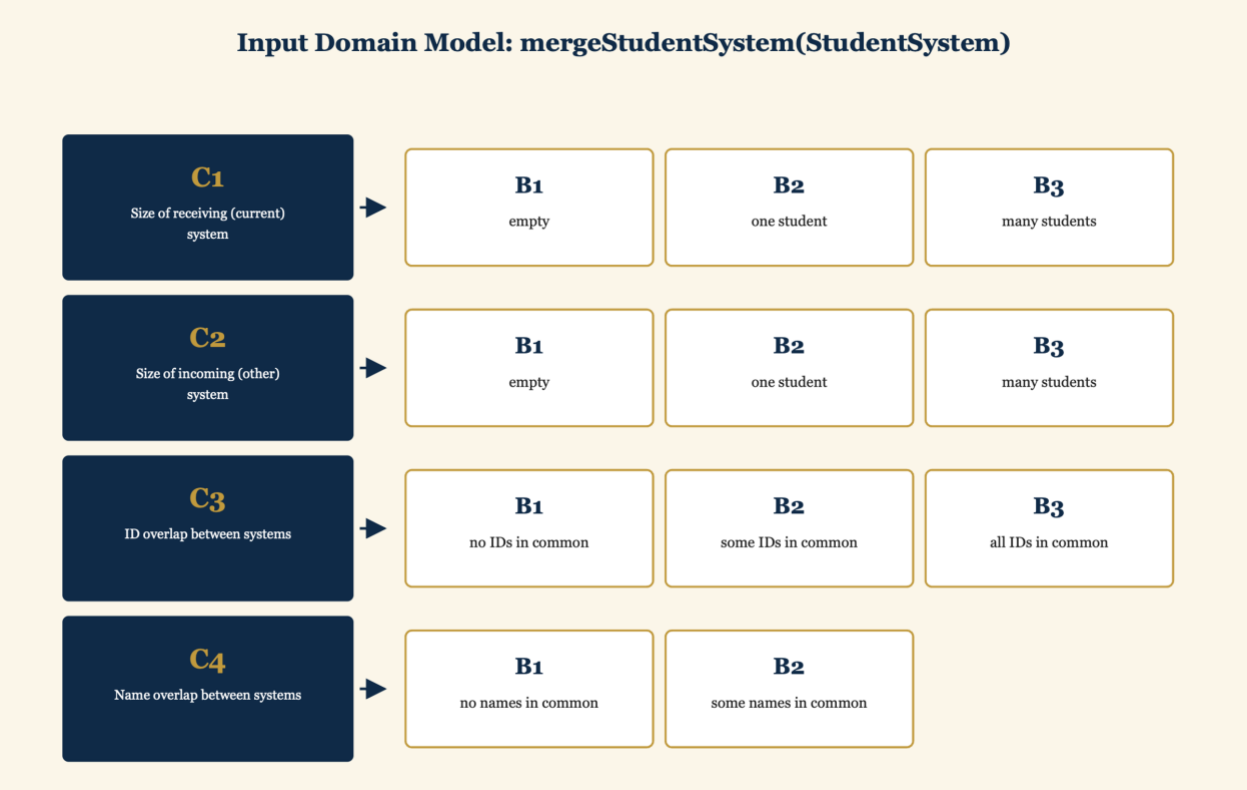
mergeStudentSystem merges a second StudentSystem into the receiver. Each incoming student is rejected if its ID or name already exists. The result depends on the receiver's state and the overlap between the two systems, not only on the parameter.

▪ Source code

```
public void mergeStudentSystem(StudentSystem other) {
    int added = 0;
    for (Student incoming : other.getStudentList()) {
        boolean uniqueID   = addUniqueID(incoming.ID);
        boolean uniqueName = addUniqueName(incoming.name);
        if (uniqueID && uniqueName) {
            this.studentList.add(incoming);
            added++;
        } else if (!uniqueID) {
            System.out.println("Rejected: non-unique ID " + incoming.ID);
        } else {
            System.out.println("Rejected: non-unique name " + incoming.name);
        }
    }
    if (added == 0) System.out.println("No new students were added.");
    else            System.out.println(added + " student(s) merged successfully.");
}
```

▪ Input Domain Model (IDM)

Four characteristics describe what can vary between two calls. Each is split into disjoint blocks. The diagram below summarises the model.



Char.	Description	Blocks
C1	Size of the receiving (current) system	B1: empty | B2: one student | B3: many students
C2	Size of the incoming (other) system	B1: empty | B2: one student | B3: many students
C3	ID overlap between the two systems	B1: none | B2: some | B3: all
C4	Name overlap between the two systems	B1: none | B2: some

Criterion 1 Each Choice Coverage (ECC)

ECC requires every block of every characteristic to appear in at least one test. The minimum is three (the largest block count). We use five to also cover combinations like C1=B2 with C3=B2.

ECC Test Requirements (block tuple per test case)

ECC requires every block of every characteristic to appear in at least one test. The block tuple chosen for each test case is shown below. Coverage check: C1 uses B1, B2, B3; C2 uses B1, B2, B3; C3 uses B1, B2, B3; C4 uses B1, B2. Every block of every characteristic appears at least once across the five tests, so ECC is satisfied.

Test Case ID	C1	C2	C3	C4	Block-tuple summary
TC-ISP-ECC-01	B2	B3	B1	B1	1 main + 3 fresh incoming, no overlap
TC-ISP-ECC-02	B2	B1	—	—	1 main, incoming empty
TC-ISP-ECC-03	B1	B2	B1	B1	Main empty, 1 fresh incoming
TC-ISP-ECC-04	B3	B3	B3	B2	All IDs and names overlap
TC-ISP-ECC-05	B2	B3	B2	B1	1 main + 3 incoming, one ID collision

Test Cases (ECC)

Test Case ID	Input	Expected Output	Actual Output	Result
TC-ISP-ECC-01	main has 1 student (Solo, ID 700); incoming has 3 fresh students (no overlap on ID or name)	All 3 incoming students added (size = 4); "merged successfully" message	List size becomes 4; success message printed	Pass
TC-ISP-ECC-02	main has 1 student (Solo, ID 750); incoming = empty	No change (size = 1); "No new students were added" message	List unchanged (size = 1); no-op message printed	Pass
TC-ISP-ECC-03	main = empty; incoming has 1 fresh student (Tariq, ID 800)	1 student added (size = 1); "Added successfully" message	List size becomes 1; success message printed	Pass
TC-ISP-ECC-04	main has 3 students; incoming has the same 3 IDs and the same 3 names	Nothing merged; "No new students" message	List unchanged; nothing merged	Pass
TC-ISP-ECC-05	main has 1 student (Solo, ID 600); incoming has 3 students, one of which reuses ID 600	Two new students added (size = 3); one rejected for non-unique ID	List size becomes 3; "non-unique ID" printed	Pass

Criterion 2 Base Choice Coverage (BCC)

BCC picks one base block per characteristic, runs that base, then varies one characteristic at a time. Our base is the typical call: many students on each side with no overlap. Four single-characteristic variations bring the total to five tests.

Characteristic	Base block	Reasoning
C1 (main size)	B3 (many)	A populated receiver is the typical use case
C2 (incoming size)	B3 (many)	A populated incoming system makes the merge meaningful
C3 (ID overlap)	B1 (none)	Disjoint IDs let every incoming student be added
C4 (name overlap)	B1 (none)	Disjoint names let every incoming student be added

BCC Test Requirements (block tuple per test case)

BCC starts from a base tuple (C1=B3, C2=B3, C3=B1, C4=B1) and then varies one characteristic at a time. The base test is BCC-01 (all base blocks, shown in bold). Tests BCC-02 through BCC-05 each flip exactly one characteristic to a non-base block (highlighted in gold) while keeping the other three at their base values. This gives the required 1 base + (B_1-1) + (B_2-1) + (B_3-1) + (B_4-1) = 1 + 2 + 2 + 2 + 1 = 8 tests in the worst case; we use 5 here because we vary each characteristic by a single representative non-base block.

Test Case ID	C1	C2	C3	C4	Variation
TC-ISP-BCC-01	**B3**	**B3**	**B1**	**B1**	Base test: all base blocks
TC-ISP-BCC-02	**B1**	B3	B1	B1	Vary C1 (main → empty)
TC-ISP-BCC-03	B3	**B1**	B1	B1	Vary C2 (incoming → empty)
TC-ISP-BCC-04	B3	B3	**B3**	B1	Vary C3 (IDs → all overlap)
TC-ISP-BCC-05	B3	B3	B1	**B2**	Vary C4 (names → some overlap)

Test Cases (BCC)

Test Case ID	Input	Expected Output	Actual Output	Result
TC-ISP-BCC-01	Base case: main has 3 students; incoming has 3 disjoint students (no overlap on ID or name)	All 3 added; "merged successfully" message	List size becomes 6; success message printed	Pass
TC-ISP-BCC-02	Vary C1 only — main empty; incoming has 3 disjoint students	All 3 added; "Added successfully" message	List size becomes 3; success message printed	Pass
TC-ISP-BCC-03	Vary C2 only — main has 3 students; incoming empty	No change; "No new students" message	List unchanged; no-op message printed	Pass
TC-ISP-BCC-04	Vary C3 only — incoming reuses every ID already in main; names are different	Nothing added; "non-unique ID" message	List unchanged; "non-unique ID" printed	Pass
TC-ISP-BCC-05	Vary C4 only — incoming has fresh IDs; one name (Alpha) collides with main	Two students added, one rejected on name	List size becomes 5; "non-unique name" printed	Pass

4.3 Graph Coverage on removeStudentByID(int ID)

▪ Function under test

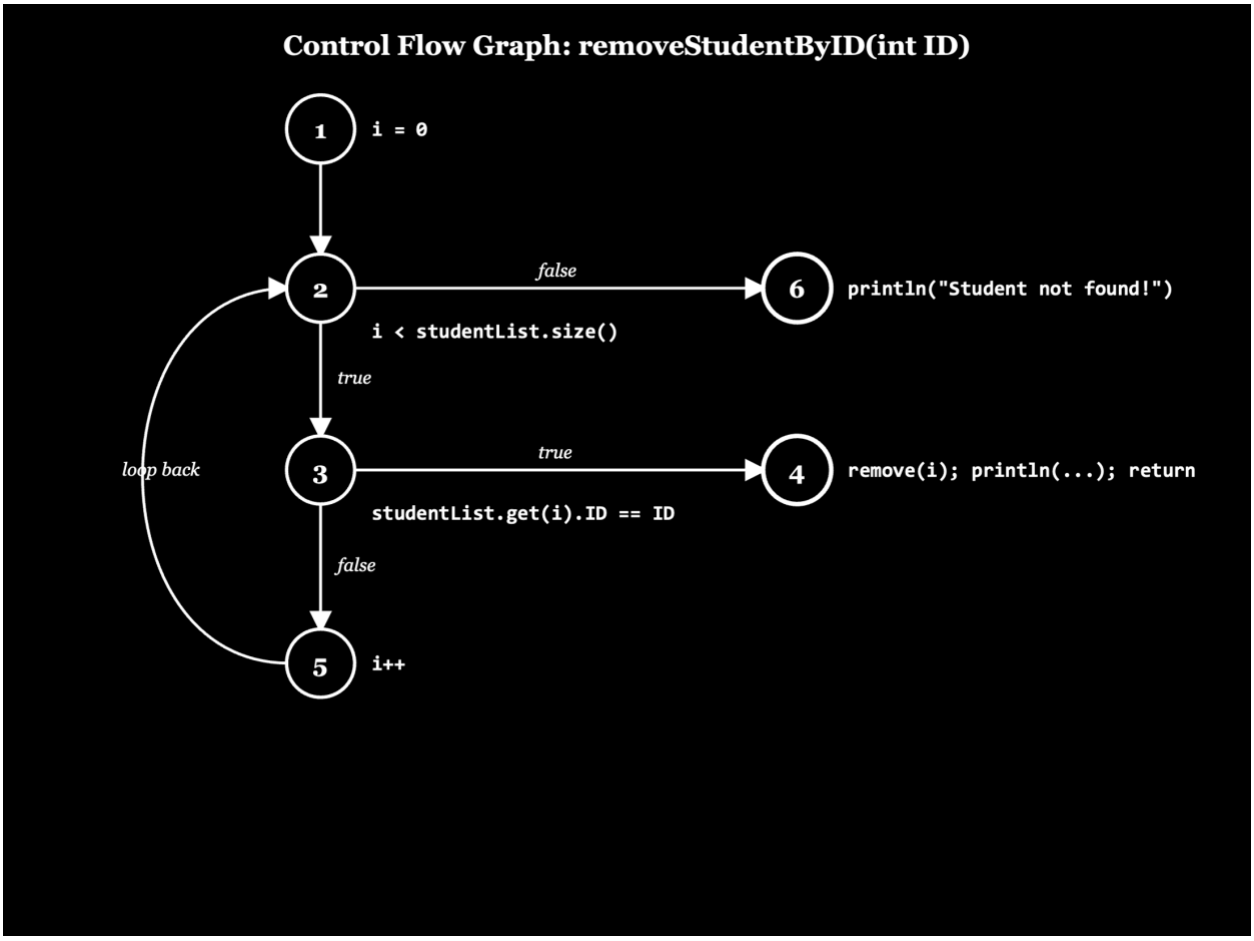
removeStudentByID walks the student list with a for-loop. On a matching ID, it removes the record, prints a success message, and returns. If the loop ends without a match, it prints a not-found message. The loop plus early return is what makes prime path coverage interesting here.

▪ Source code

```
public void removeStudentByID(int ID) {
    for (int i = 0; i < this.studentList.size(); i++) {
        if (this.studentList.get(i).ID == ID) {
            this.studentList.remove(i);
            System.out.println("Student removed successfully!");
            return;
        }
    }
    System.out.println("Student not found!");
}
```

▪ Control Flow Graph

Using the standard for-loop CFG convention, the initialiser, condition, and increment are split into separate nodes. This keeps the loop-back edge explicit.



Set of nodes:

```
N = { 1, 2, 3, 4, 5, 6 }
```

Set of edges:

```
E = { (1,2), (2,3), (2,6), (3,4), (3,5), (5,2) }
```

Initial node: 1 · Final nodes: { 4, 6 }

Criterion 1 Node Coverage (NC)

NC requires every node to be reached by at least one test path. The five cases below visit all of {1,2,3,4,5,6}, including both final nodes (success and not-found).

NC Test Requirements (path walked and nodes visited per test case)

NC requires every node in N = {1, 2, 3, 4, 5, 6} to be reached by at least one test path. The path each test walks and the nodes it visits are listed below. Coverage check: across the five paths the union of visited nodes is {1, 2, 3, 4, 5, 6}, so every node is reached and NC is satisfied. Both final nodes are reached as well — node 4 by NC-02, NC-03, NC-05 (success exit) and node 6 by NC-01, NC-04 (not-found exit).

Test Case ID	Path walked	Nodes covered
TC-GC-NC-01	1 → 2 → 6	{1, 2, 6}
TC-GC-NC-02	1 → 2 → 3 → 5 → 2 → 3 → 5 → 2 → 3 → 4	{1, 2, 3, 4, 5}
TC-GC-NC-03	1 → 2 → 3 → 5 → 2 → 3 → 5 → 2 → 3 → 5 → 2 → 3 → 4	{1, 2, 3, 4, 5}
TC-GC-NC-04	1 → 2 → 3 → 5 → 2 → 3 → 5 → 2 → 3 → 5 → 2 → 3 → 5 → 2 → 6	{1, 2, 3, 5, 6}
TC-GC-NC-05	1 → 2 → 3 → 5 → 2 → 3 → 5 → 2 → 3 → 5 → 2 → 3 → 5 → 2 → 3 → 4	{1, 2, 3, 4, 5}

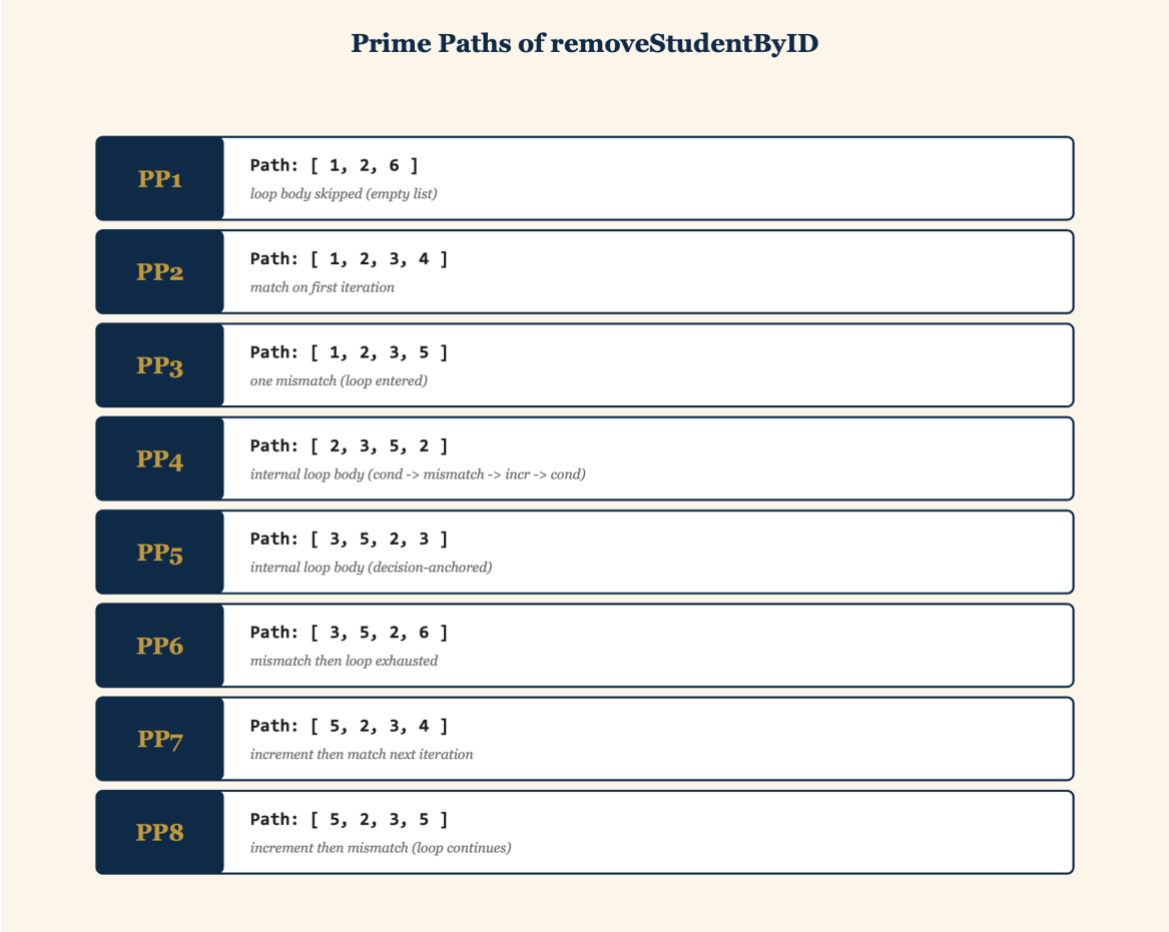
Test Cases (NC)

Test Case ID	Input	Expected Output	Actual Output	Result
TC-GC-NC-01	List = [Aisha 100, Bilal 101]; remove ID = 9999	List unchanged (size = 2); "Student not found!" printed	Nothing removed; “Student not found!” printed	Pass
TC-GC-NC-02	List = [Cyrus 200, Dina 201, Ehab 202]; remove ID = 202	Ehab removed (size = 2); "removed successfully" printed	Ehab removed; success message printed	Pass
TC-GC-NC-03	List = [Fatima 300, Ghada 301, Hadi 302, Iman 303]; remove ID = 303	Iman removed (size = 3); success message printed	Iman removed (size = 3); “Student removed successfully!” printed	Pass
TC-GC-NC-04	List = [Jude 400, Khaled 401, Layla 402, Mira 403]; remove ID = 9999	List unchanged (size = 4); "Student not found!" printed	List unchanged (size = 4); “Student not found!” printed	Pass
TC-GC-NC-05	List = [Nora 500, Omar 501, Pari 502, Qasim 503, Rana 504]; remove ID = 504	Rana removed (size = 4); success message printed	Rana removed (size = 4); “Student removed successfully!” printed	Pass

Criterion 2 Prime Path Coverage (PPC)

PPC requires every prime path to be toured by at least one test path. A prime path is a simple path that is not a sub-path of a longer simple path. A simple path has no repeated nodes, except the first and last may match, so a closed loop is one simple path.

We listed all simple paths in length order and kept only those not contained in a longer one. This gives eight prime paths: three start at node 1 and walk into the loop, and five are internal loop fragments starting at nodes 2, 3, or 5.



The diagram above lists the eight prime paths themselves (properties of the graph). The table below shows the five test paths that we run, each of which tours one or more of those prime paths. One test path can tour several prime paths because a test path may contain internal loops, which a prime path cannot. For example, 1 → 2 → 3 → 5 → 2 → 6 tours PP3, PP4, and PP6 at once.

Test Path ID	Test path Description	Prime paths toured
TC-GC-PPC-01	1 → 2 → 6	PP1
TC-GC-PPC-02	1 → 2 → 3 → 4	PP2
TC-GC-PPC-03	1 → 2 → 3 → 5 → 2 → 6	PP3, PP4, PP6
TC-GC-PPC-04	1 → 2 → 3 → 5 → 2 → 3 → 4	PP3, PP4, PP5, PP7
TC-GC-PPC-05	1 → 2 → 3 → 5 → 2 → 3 → 5 → 2 → 6	PP3, PP4, PP5, PP6, PP8

Coverage check: across the five test paths, every prime path PP1 through PP8 is toured at least once, so PPC is satisfied.

Test Cases (PPC)

Test Case ID	Input	Expected Output	Actual Output	Result
TC-GC-PPC-01	Empty list; remove ID = 123	Loop body never executes (PP1); not-found message printed; list size = 0	List size = 0; “Student not found!” printed	Pass
TC-GC-PPC-02	List = [Ali, ID 555]; remove ID = 555	Match on first iteration (PP2); Ali removed; success message	Ali removed (size = 0); “Student removed successfully!” printed	Pass
TC-GC-PPC-03	List = [Hadi, ID 700]; remove ID = 8888	One mismatch then loop exits (tours PP3, PP4, PP6); list unchanged; not-found	List unchanged (size = 1); “Student not found!” printed	Pass
TC-GC-PPC-04	List = [Iman ID 800, Jude ID 801]; remove ID = 801	One mismatch then match on iteration 2 (tours PP3, PP4, PP5, PP7); Jude removed; success	Jude removed (size = 1); “Student removed successfully!” printed	Pass
TC-GC-PPC-05	List = [K, L, M] (IDs 900, 901, 902); remove ID = 9999	Three iterations then exit — loop body executes more than once (tours PP3, PP4, PP5, PP6, PP8); list unchanged; not-found	List unchanged (size = 3); “Student not found!” printed	Pass

5. Introduction to the Tool

5.1 Tool Description

We used JUnit 5 (Jupiter) for all tests. It is the standard Java unit-testing framework and one of the tools listed in the project description. Each Section 4 test case was written as a Java method so the behaviour could be checked by running the code.

Features of JUnit 5 we relied on:

- @Test, @BeforeEach, and @DisplayName annotations to declare and label individual tests.
- Assertions library (assertEquals, assertTrue, assertFalse, assertNotNull) for verifying outcomes.
- Standard-output capture via System.setOut and ByteArrayOutputStream so we can assert on the messages StudentSystem prints.
- Per-test isolation via @BeforeEach, which creates a fresh StudentSystem for every test so state cannot leak between them.
- Clear pass/fail reporting in the IDE so each test case ID in Section 4 matches a row in the runner.

5.2 Tool Setup and Configuration

The tests live in a /tests folder next to the original code. We did not modify the source. Tests can be run from an IDE or the terminal.

6. Tool Usage

6.1 Application of the Tool

The thirty test cases of Section 4 became thirty JUnit methods in three classes (one per model). Each method follows the same pattern: @BeforeEach creates a fresh StudentSystem, the test adds the input, calls the function, and asserts on both the list state and the captured console output.

Test class file	Function under test	Tests
LogicCoverageTest.java	displayTop5()	10 (PC × 5, CACC × 5)
InputSpacePartitioningTest.java	mergeStudentSystem(StudentSystem)	10 (ECC × 5, BCC × 5)
GraphCoverageTest.java	removeStudentByID(int)	10 (NC × 5, PPC × 5)
TOTAL	—	30

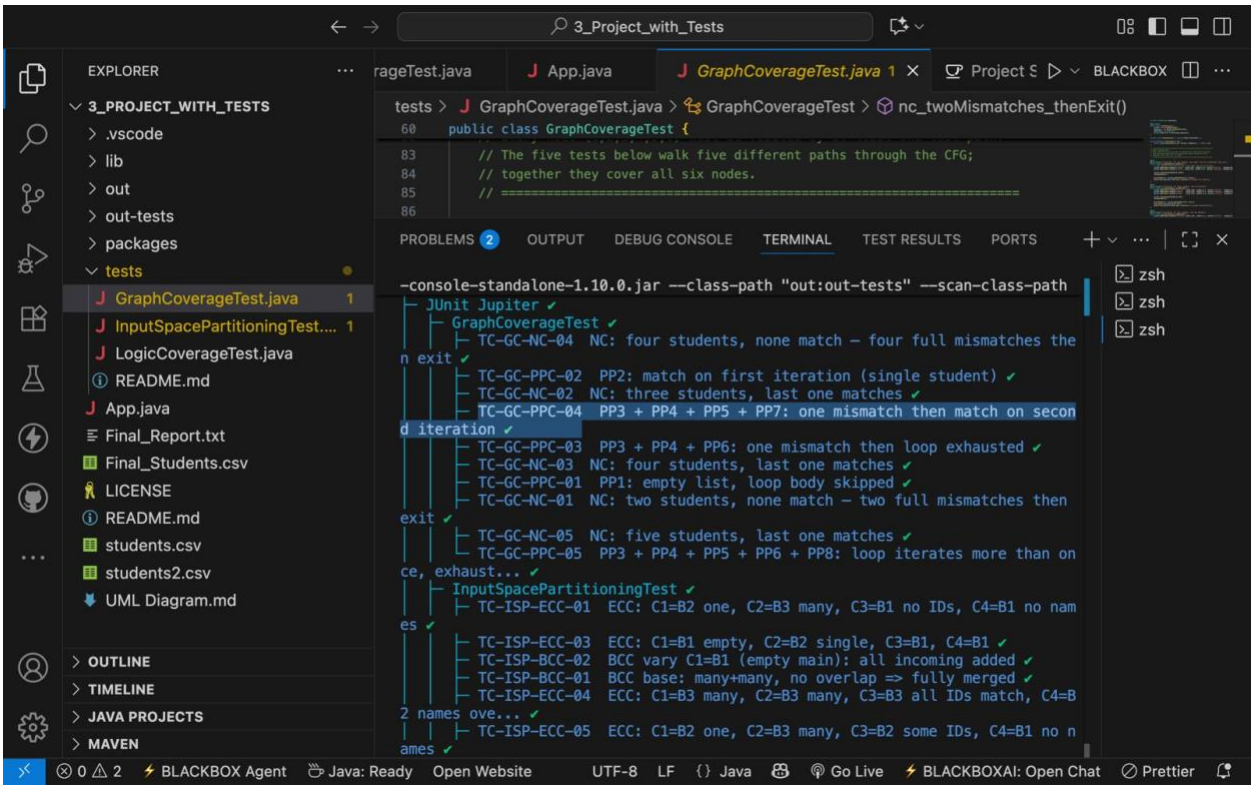
6.2 Example Test Case Execution

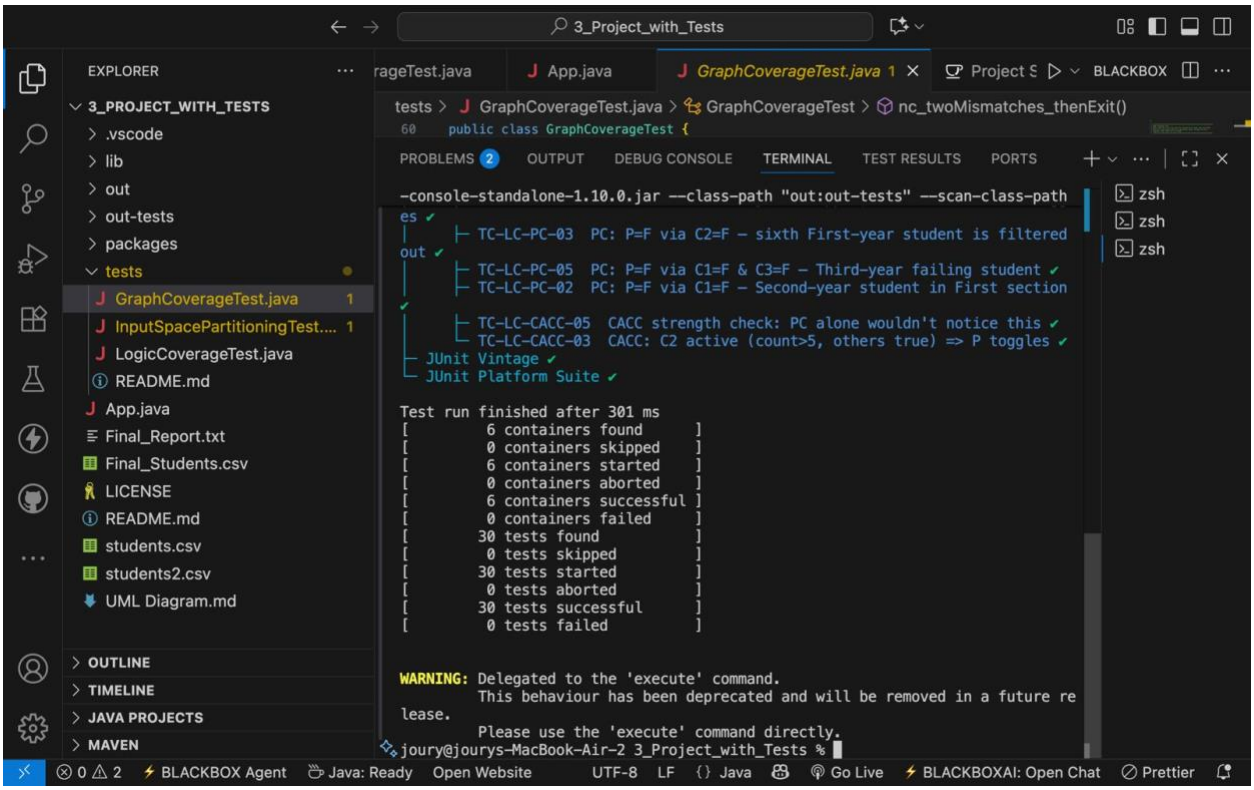
The snippet below implements TC-GC-PPC-04 (the test that touches the loop's back-edge). Two students are added so the second one matches; removeStudentByID is called with that ID; assertions check the list size, ID membership, and success message.

```
@Test
@DisplayName("TC-GC-PPC-04  PP3 + PP4 + PP5 + PP7: one mismatch then match on second
iteration")
public void ppc_PP4_secondIterationMatches() {
    system.addStudent("Iman", 800, 3.0, "First",  "CS", true);   // index 0
    system.addStudent("Jude", 801, 3.5, "Second", "IS", true);   // index 1

    system.removeStudentByID(801);
    releaseStdout();

    assertEquals(1, system.getStudentList().size());
    assertTrue (containsId(800));
    assertFalse(containsId(801));
    assertTrue (captured.toString().contains("removed successfully"));
}
```


3_Project_with_Tests
EXPLORER
3_PROJECT_WITH_TESTS
.vscode
lib
out
out-tests
packages
tests
GraphCoverageTest.java
InputSpacePartitioningTest....
LogicCoverageTest.java
README.md
App.java
Final_Report.txt
Final_Students.csv
LICENSE
README.md
students.csv
students2.csv
UML Diagram.md
OUTLINE
TIMELINE
JAVA PROJECTS
MAVEN
App.java
GraphCoverageTest.java 1
BLACKBOX
tests > GraphCoverageTest.java > GraphCoverageTest > nc_twoMismatches_thenExit()
60 public class GraphCoverageTest {
83 // The five tests below walk five different paths through the CFG;
84 // together they cover all six nodes.
85 // ==
PROBLEMS 2
OUTPUT
DEBUG CONSOLE
TERMINAL
TEST RESULTS
PORTS
zsh
zsh
zsh
-console-standalone-1.10.0.jar --class-path "out:out-tests" --scan-class-path
JUnit Jupiter ✔
GraphCoverageTest ✔
TC-GC-NC-04 NC: four students, none match — four full mismatches then exit ✔
TC-GC-PPC-02 PP2: match on first iteration (single student) ✔
TC-GC-NC-02 NC: three students, last one matches ✔
TC-GC-PPC-04 PP3 + PP4 + PP5 + PP7: one mismatch then match on second iteration ✔
TC-GC-PPC-03 PP3 + PP4 + PP6: one mismatch then loop exhausted ✔
TC-GC-NC-03 NC: four students, last one matches ✔
TC-GC-PPC-01 PP1: empty list, loop body skipped ✔
TC-GC-NC-01 NC: two students, none match — two full mismatches then exit ✔
TC-GC-NC-05 NC: five students, last one matches ✔
TC-GC-PPC-05 PP3 + PP4 + PP5 + PP6 + PP8: loop iterates more than once, exhaust... ✔
InputSpacePartitioningTest ✔
TC-ISP-ECC-01 ECC: C1=B2 one, C2=B3 many, C3=B1 no IDs, C4=B1 no names ✔
TC-ISP-ECC-03 ECC: C1=B1 empty, C2=B2 single, C3=B1, C4=B1 ✔
TC-ISP-BCC-02 BCC vary C1=B1 (empty main): all incoming added ✔
TC-ISP-BCC-01 BCC base: many+many, no overlap => fully merged ✔
TC-ISP-ECC-04 ECC: C1=B3 many, C2=B3 many, C3=B3 all IDs match, C4=B2 names ove... ✔
TC-ISP-ECC-05 ECC: C1=B2 one, C2=B3 many, C3=B2 some IDs, C4=B1 no names ✔
0 ⚠ 2
BLACKBOX Agent
Java: Ready
Open Website
UTF-8
LF
{} Java
Go Live
BLACKBOXAI: Open Chat
Prettier


3_Project_with_Tests
EXPLORER
3_PROJECT_WITH_TESTS
.vscode
lib
out
out-tests
packages
tests
GraphCoverageTest.java
InputSpacePartitioningTest....
LogicCoverageTest.java
README.md
App.java
Final_Report.txt
Final_Students.csv
LICENSE
README.md
students.csv
students2.csv
UML Diagram.md
OUTLINE
TIMELINE
JAVA PROJECTS
MAVEN
App.java
GraphCoverageTest.java 1
BLACKBOX
tests > GraphCoverageTest.java > GraphCoverageTest > nc_twoMismatches_thenExit()
60 public class GraphCoverageTest {
PROBLEMS 2
OUTPUT
DEBUG CONSOLE
TERMINAL
TEST RESULTS
PORTS
zsh
zsh
zsh
-console-standalone-1.10.0.jar --class-path "out:out-tests" --scan-class-path
TC-LC-PC-03 PC: P=F via C2=F — sixth First-year student is filtered out ✔
TC-LC-PC-05 PC: P=F via C1=F & C3=F — Third-year failing student ✔
TC-LC-PC-02 PC: P=F via C1=F — Second-year student in First section ✔
TC-LC-CACC-05 CACC strength check: PC alone wouldn't notice this ✔
TC-LC-CACC-03 CACC: C2 active (count>5, others true) => P toggles ✔
JUnit Vintage ✔
JUnit Platform Suite ✔
Test run finished after 301 ms
[6 containers found]
[0 containers skipped]
[6 containers started]
[0 containers aborted]
[6 containers successful]
[0 containers failed]
[30 tests found]
[0 tests skipped]
[30 tests started]
[0 tests aborted]
[30 tests successful]
[0 tests failed]
WARNING: Delegated to the 'execute' command.
This behaviour has been deprecated and will be removed in a future release.
Please use the 'execute' command directly.
joury@jourys-MacBook-Air-2 3_Project_with_Tests %
0 ⚠ 2
BLACKBOX Agent
Java: Ready
Open Website
UTF-8
LF
{} Java
Go Live
BLACKBOXAI: Open Chat
Prettier

7. Testing Results

7.1 Total Number of Tests

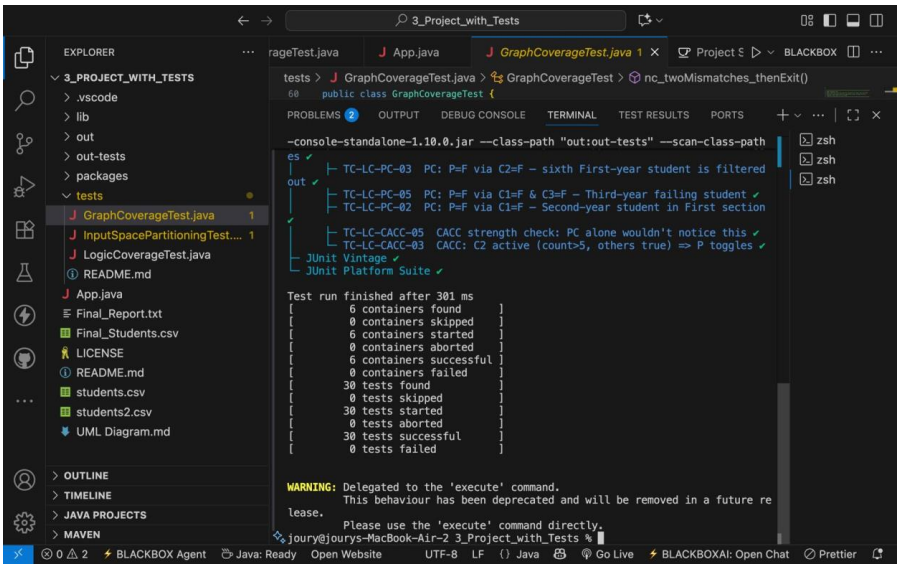
Thirty JUnit test methods were executed across the three classes: five per criterion, two criteria per model, three models.

7.2 Pass / Fail Results

Summary of the run, broken down by coverage criterion:

Model	Coverage Criterion	Total	Passed	Failed	Pass %
Logic	Predicate Coverage (PC)	5	5	0	100%
Logic	Correlated Active Clause Coverage (CACC)	5	5	0	100%
ISP	Each Choice Coverage (ECC)	5	5	0	100%
ISP	Base Choice Coverage (BCC)	5	5	0	100%
Graph	Node Coverage (NC)	5	5	0	100%
Graph	Prime Path Coverage (PPC)	5	5	0	100%
TOTAL	—	30	30	0	100%

All thirty tests passed. The implementation produced the expected output for every input prescribed by Section 4.



7.3 Details of Failures

There were no failures. Every test produced its expected output.

8. Conclusion

8.1 What the Tool Found

JUnit 5 confirmed that the three functions behave as the chosen criteria predict. Each test case maps to one JUnit method, so the IDE's run report lines up row-for-row with the Section 4 tables. Every criterion we targeted was satisfied.

Writing the tests taught us that turning prime paths into actual inputs forces careful thinking about the order students are added, since that order decides which iteration matches. Edge cases (empty list, one mismatch, three in a row) all behaved as the CFG predicted.

8.2 Challenges Faced

- Several StudentSystem methods print to System.out instead of returning a value. We redirected System.out into a ByteArrayOutputStream in @BeforeEach to capture the text, and restored it afterwards.
- The Student class fields are public and there is no equals() override, so the tests had to compare individual fields rather than rely on object equality.
- displayTop5() prints all four years into one buffer, which made asserting on a single year tricky. We sliced the captured output between the year headers ("First Year:", "Second Year:", etc.) and asserted on the relevant slice only.
- Enumerating prime paths by hand is easy to get wrong when the graph has both an early return and a loop-back edge. We followed the algorithm step by step (listing simple paths by length, then discarding sub-paths) to confirm the eight prime paths in Section 4.3.

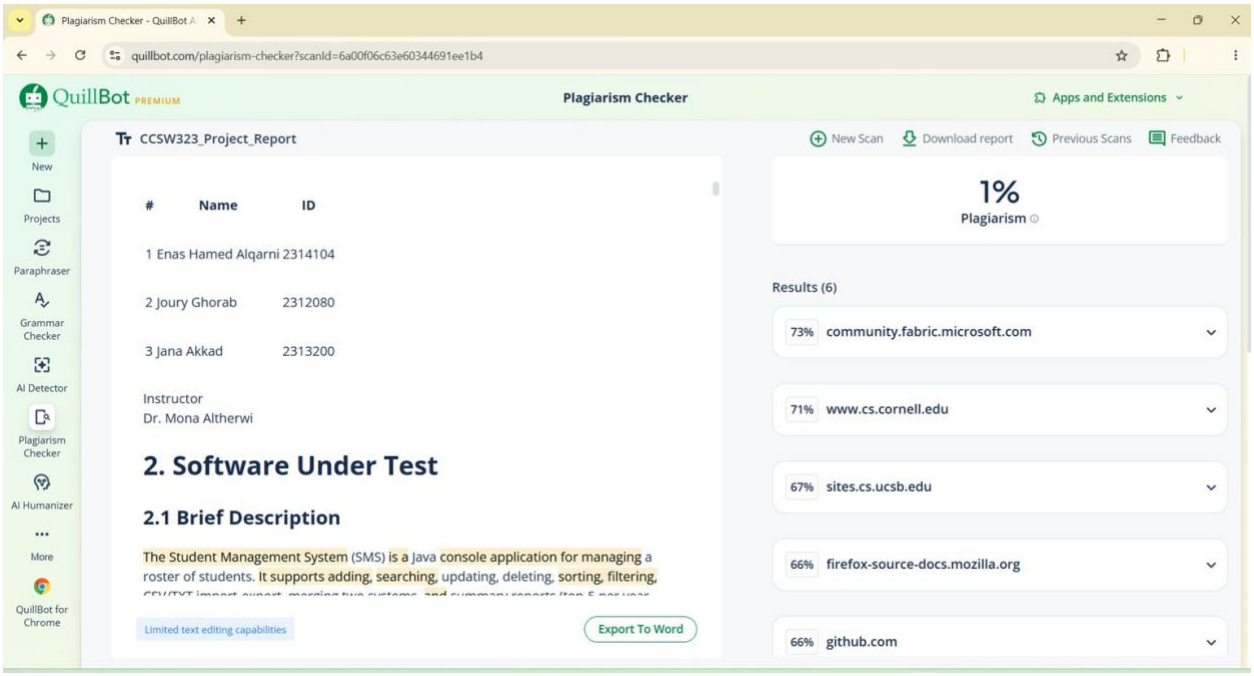
8.3 Recommendations

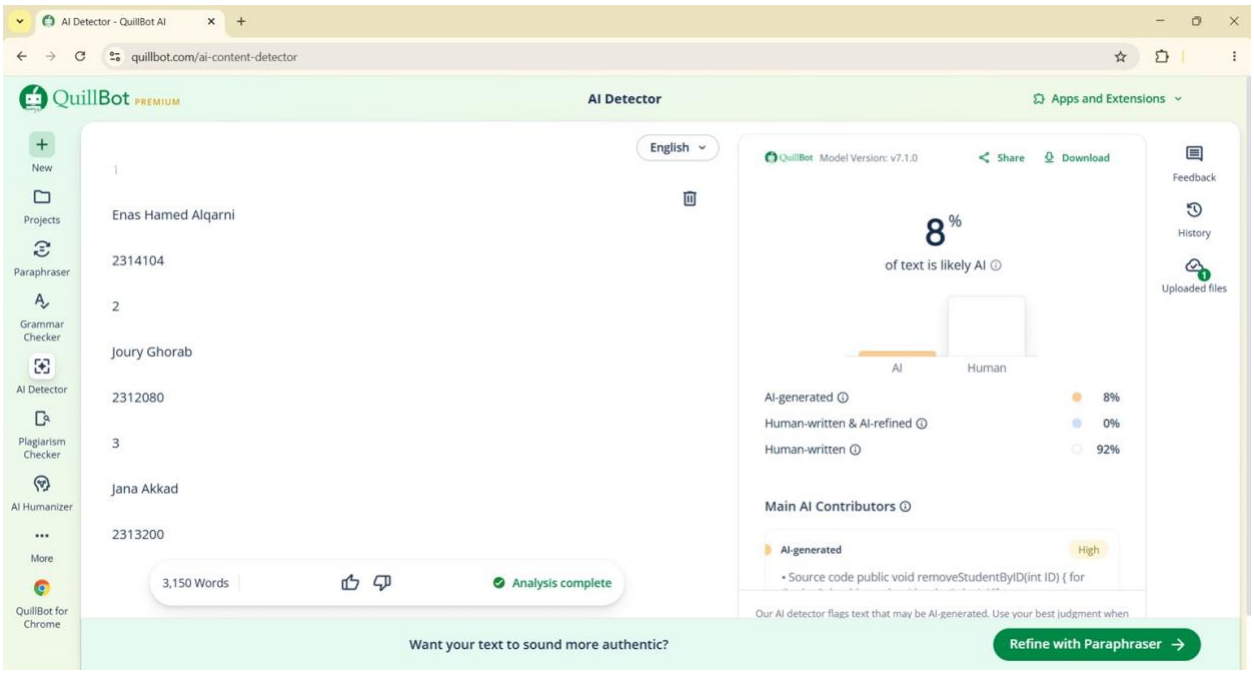
- Refactor StudentSystem methods to return result objects or throw exceptions instead of printing directly, so assertions can check return values rather than scraping stdout.
- Override equals and hashCode in the Student class so that ArrayList.contains and assertEquals work naturally on Student objects.
- Add the tests to a Maven or Gradle build and wire them into CI so any regression is caught at commit time.

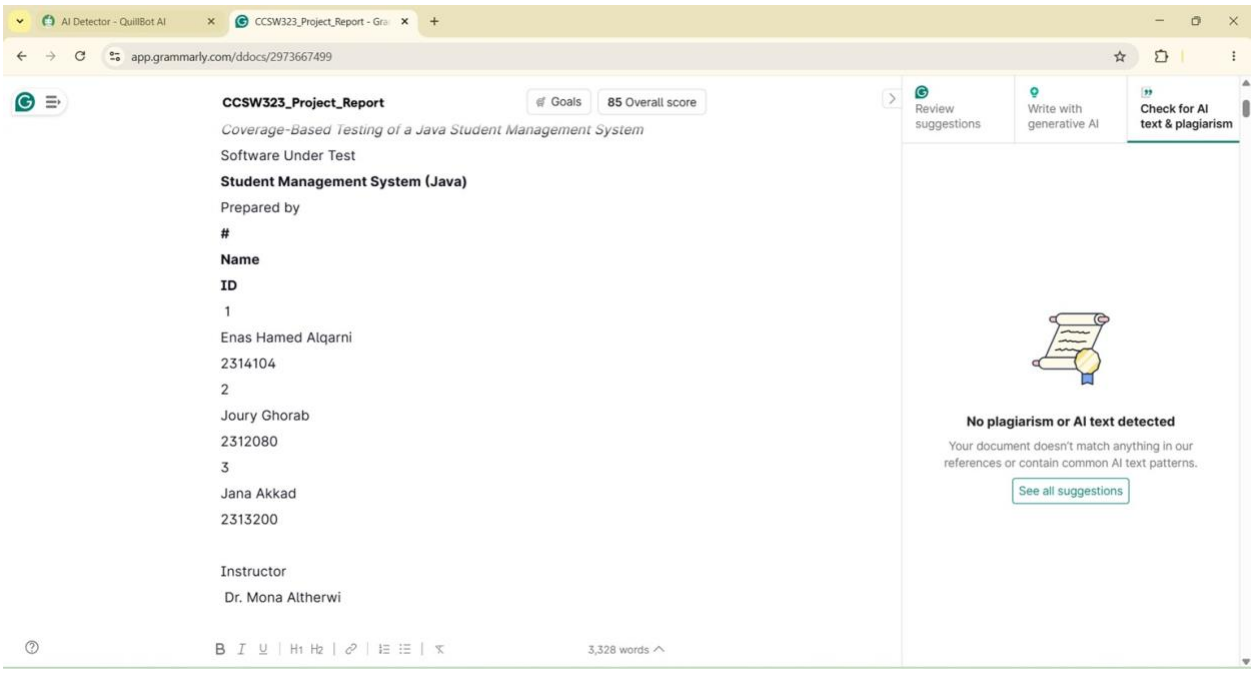
Using three coverage models on the same project showed that each produces a different style of test: Logic Coverage on predicate clauses, ISP on input characteristics and blocks, and PPC on walking the CFG. Together they produced thirty unique test cases, all passing on the current implementation.

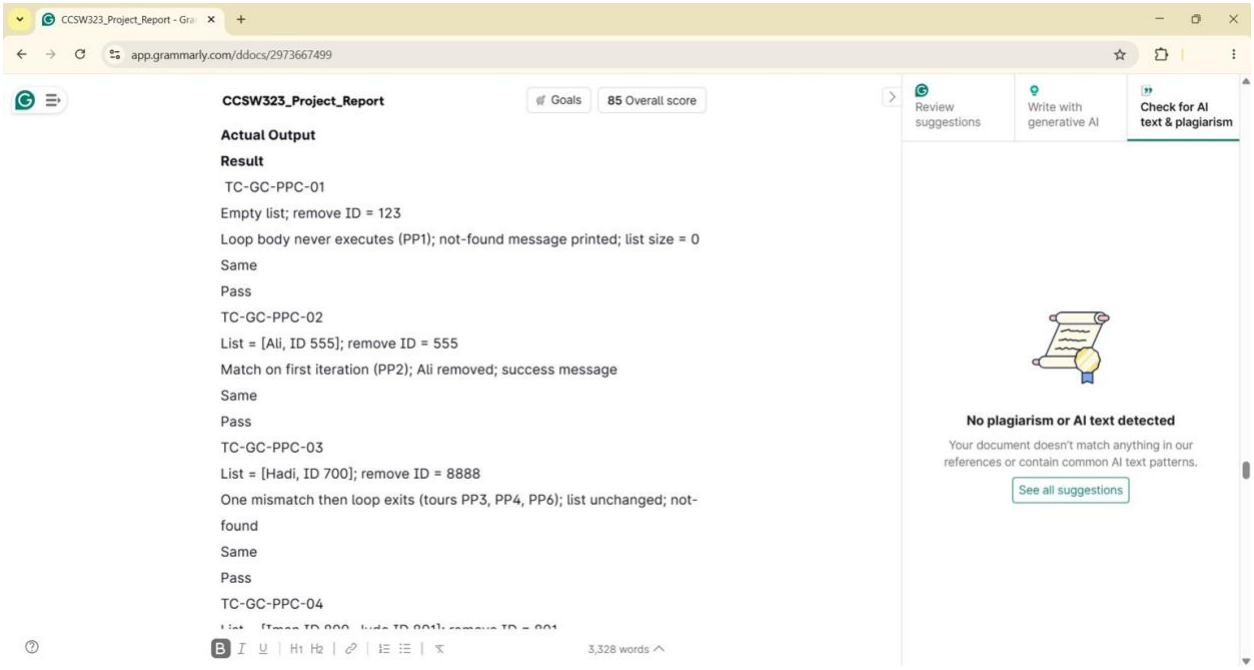
9. Plagiarism and AI-Detection Report

The report was checked for plagiarism and AI-generated content before submission. The Turnitin similarity and AI-detection results are shown below.






AI Detector - QuillBot AI
CCSW323_Project_Report - Gra
app.grammarly.com/ddocs/2973667499
CCSW323_Project_Report
Goals
85 Overall score
Review suggestions
Write with generative AI
Check for AI text & plagiarism
Coverage-Based Testing of a Java Student Management System
Software Under Test
Student Management System (Java)
Prepared by
#
Name
ID
1
Enas Hamed Alqarni
2314104
2
Joury Ghorab
2312080
3
Jana Akkad
2313200
Instructor
Dr. Mona Altherwi
No plagiarism or AI text detected
Your document doesn't match anything in our references or contain common AI text patterns.
See all suggestions
3,328 words


CCSW323_Project_Report - Gra
app.grammarly.com/ddocs/2973667499
CCSW323_Project_Report
Goals
85 Overall score
Review suggestions
Write with generative AI
Check for AI text & plagiarism
Actual Output
Result
TC-GC-PPC-01
Empty list; remove ID = 123
Loop body never executes (PP1); not-found message printed; list size = 0
Same
Pass
TC-GC-PPC-02
List = [Ali, ID 555]; remove ID = 555
Match on first iteration (PP2); Ali removed; success message
Same
Pass
TC-GC-PPC-03
List = [Hadi, ID 700]; remove ID = 8888
One mismatch then loop exits (tours PP3, PP4, PP6); list unchanged; not-found
Same
Pass
TC-GC-PPC-04
No plagiarism or AI text detected
Your document doesn't match anything in our references or contain common AI text patterns.
See all suggestions
3,328 words